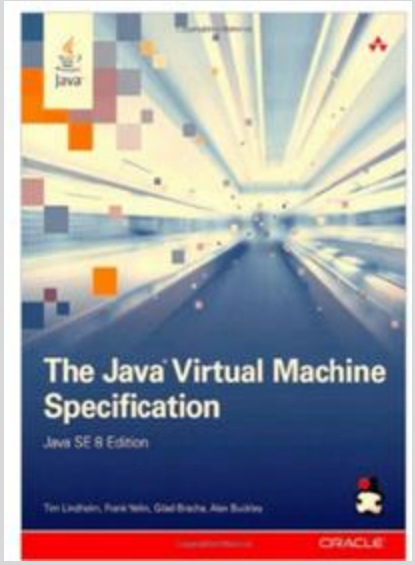

VM ANATOMY: THE JVM

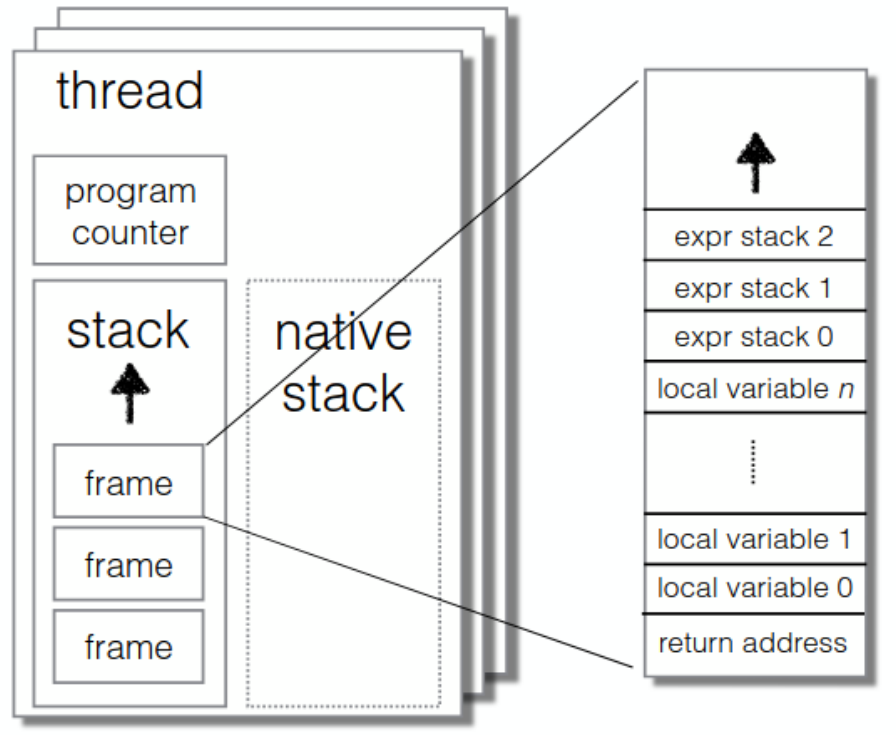
- Src: https://docs.oracle.com/javase/specs/



Components inside a VM

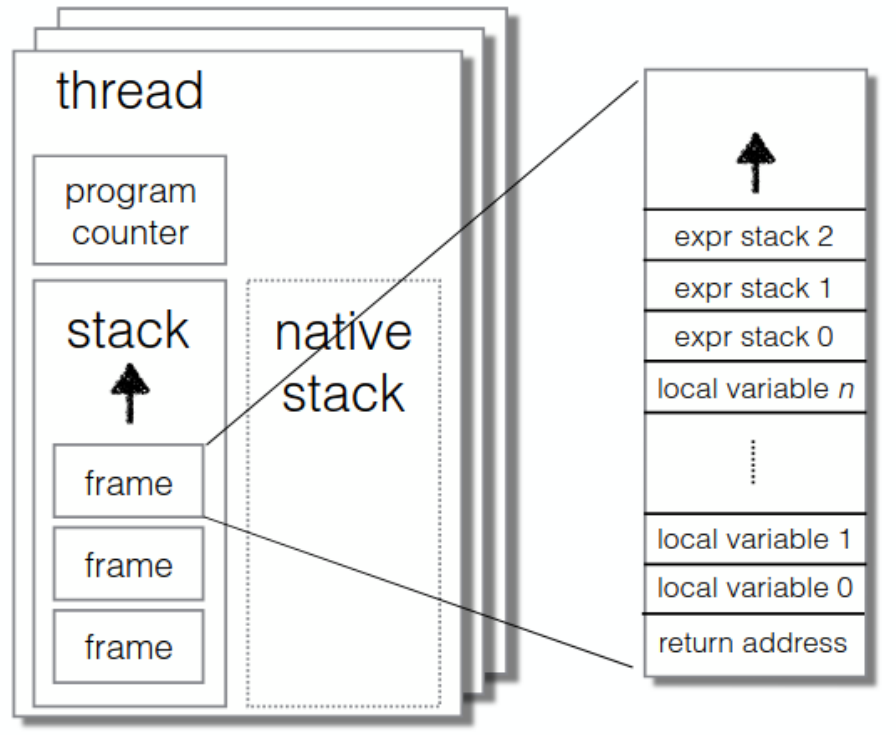
- Maintenance of state
  - Explicit state: Memory, registers
  - Implicit state: Translated code, working areas, stateful I/O, …
- Execution
  - Executing the “virtual machine instructions” with side-effects on state
- Libraries and support code to supplied required functionality
  - E.g., the equivalent of OS traps to VM-specific/language-specific functions
- External interfaces
  - OS, foreign function interfaces (FFI) to call user libraries (in other languages)

VM thread state



- **Thread:**
  - A thread in Java is like a separate worker that runs instructions independently. Each thread has its own set of resources, like a **program counter** and **stack**.
  - The JVM can have multiple threads, and each of these threads will have its own state shown in the diagram.
- **Program Counter:**
  - This is like a bookmark in the method you're currently executing. It tells the JVM which instruction (or bytecode) to run next.
  - Think of it as the line number in a textbook that shows which instruction the thread is about to execute.
- **Stack:**
  - The stack is a storage area that keeps track of all the method calls and the local variables for each thread.
  - When a thread calls a method, a **frame** is pushed onto the stack. Each frame represents a method and contains things like local variables and temporary data.
- **Native Stack:**
  - Sometimes, the JVM executes **native code**, which is code written in a language like C or C++. This native code has its own separate stack called the **native stack**.
  - The native stack is separate from the JVM's regular stack and is used when your Java program interacts with non-Java code (such as native libraries).

VM thread state

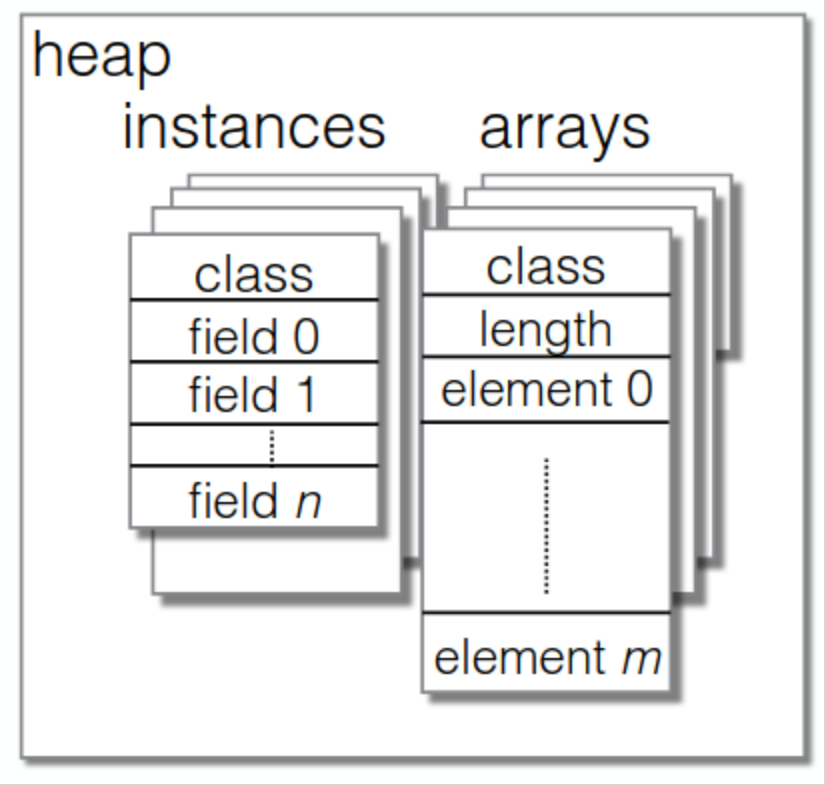


- Each **frame** in the stack contains:
  - **Local Variables:** These are the variables that the method is using. For example, in a method that adds two numbers, the local variables would be those two numbers.
  - **Expression Stack:** This is where temporary calculations are done. Think of it as a workspace for intermediate results, where operations are performed before the final result is ready.
  - **Return Address:** This is like a "return point" for the method. It tells the JVM where to go back to when the method finishes. The JVM uses the return address to continue from where the method was called.

Keeping track where in the code

- When the JVM is executing, it needs to keep track of exactly where it is in the code.
- The **program counter** or **return address** is represented as a "triple," meaning it contains three pieces of information:
  - **Current Class:** Which class is the method in? This helps the JVM understand where the method is located.
  - **Current Method:** Which method is being executed? This tells the JVM exactly which method it is running.
  - **Bytecode Offset:** The position in the bytecode (Java's compiled code) that’s currently being executed. The JVM uses this to know which instruction it should run next.

JVM object heap



- **Heap:**
  - The heap is where **all objects** are stored in Java. It is a part of the JVM memory that is used to allocate memory for objects at runtime.
  - The heap consists of two main parts: **instances** and **arrays**.
- **Instances:**
  - These are objects that belong to a particular class. For every instance of a class created (i.e., an object), a block of memory is allocated on the heap to store its **fields**.
  - **Fields** are the variables that define the state of an object (like the properties of an object).
    - For example, for a `Car` class, fields might include `color`, `make`, `model`, etc.
  - Each instance has a **class** label that identifies what type of object it is.
  - The fields (`field 0`, `field 1`, ..., `field n`) represent the data attributes of the object. These fields hold the actual data of the object.
  - **Example:**
  - A `Car` object in the heap might look like this:
    - **class**: `Car`
    - **field 0**: `color` (e.g., "Red")
    - **field 1**: `model` (e.g., "Sedan")
    - **field n**: `make` (e.g., "Toyota")
- **Arrays:**
  - Arrays are a special kind of object in Java, and they too are stored in the heap.
  - An **array** in Java is essentially an object with a **fixed length** and a series of **elements**.

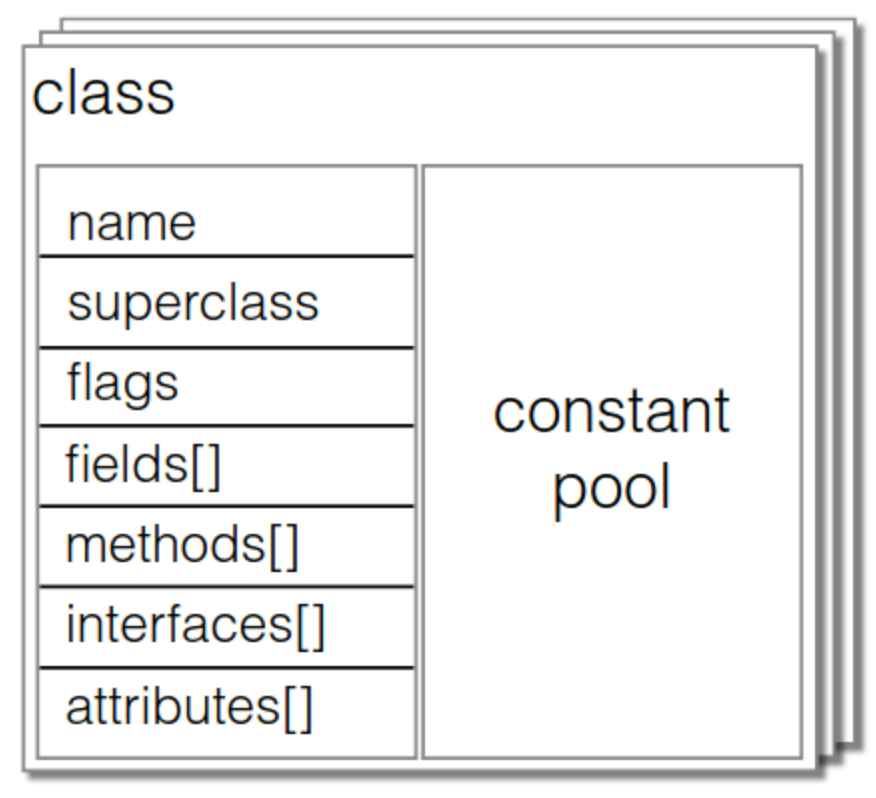
Object fields

- Field Types in Objects or Arrays : Each field in an object or element in an array can store one of two types of data:
  - A reference to another object (this is a reference type)
  - A primitive value (this is a value type)

    i) 8/16/32/64-bit integer

    ii) 32/64-bit IEEE 754 floating-point

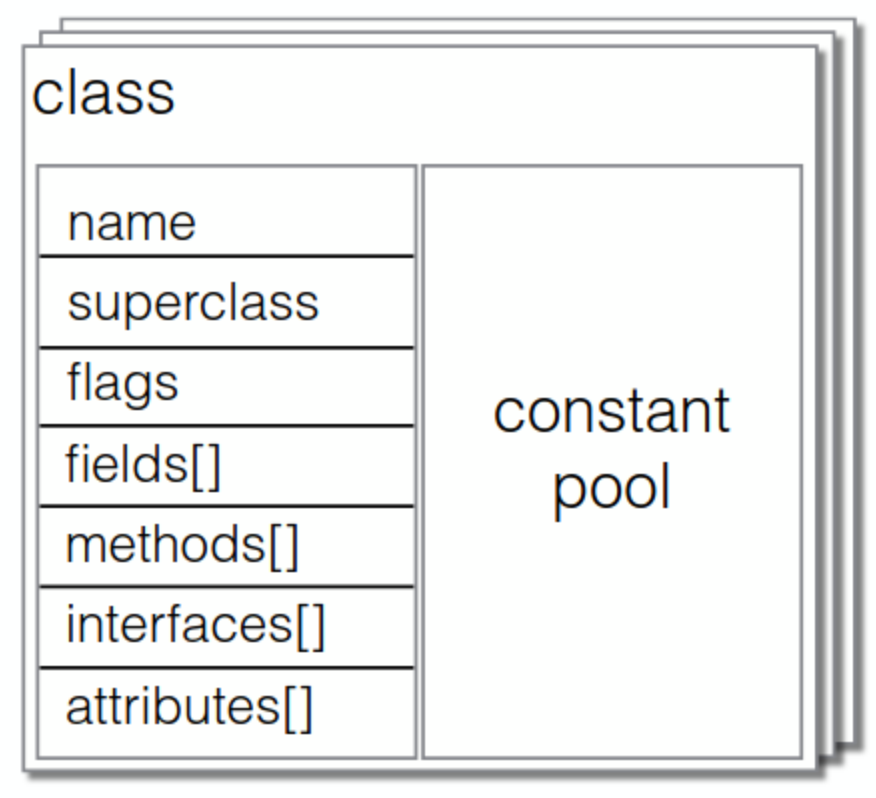
JVM classes



JVM Class Structure:

- **Name**: The name of the class, which is typically the fully qualified name including the package name (e.g., `com.example.Car`).
- **Superclass**: The class from which this class is inheriting.
- **Flags**: These are metadata flags that describe the class's properties. These include access modifiers (like `public`, `private`), whether the class is `abstract`, `final`, or if it has other special properties.
- **Fields**: An array of **fields** that define the properties or variables of the class. Each field in the class can hold data or references to other objects. For example, in a `Car` class, fields could include `model`, `color`, and `engine`.
- **Methods**: An array of **methods** that define the behavior of the class. Each method contains code that performs specific tasks when called. For example, a method in the `Car` class might be `drive()`, which describes how the car moves.
- **Interfaces**: An array of **interfaces** that the class implements. Java allows classes to implement one or more interfaces, which are collections of abstract methods that the class must define.
- **Attributes**: An array of additional **attributes** that provide extra information about the class, such as annotations or other metadata that the class might include.

JVM classes



Fields as Indexes to the Constant Pool:

- Each **field** in a class is represented as an **index** into the **constant pool**.
- Fields don't directly store their names or types; they refer to them via indexes into the constant pool.
- **Constants**:
  - These include **primitive values** like numbers (e.g., `int`, `double`) and **strings**. In Java, string literals like `"Hello, world!"` are stored in the constant pool.

  Example:
  - A string constant `"Car"` might be stored in the constant pool to represent the name of the class.

- **Structures Describing Fields, Methods, Interfaces, and Attributes**:
  - The constant pool also stores **descriptions** for the fields, methods, interfaces, and attributes that are part of the class.
  - For example, each method in the class (like `drive()`) would have a corresponding entry in the constant pool with details about the method (such as its name, parameters, and return type).

Field spec in class files

- The `field_info` structure is used to describe each field in a class. For example, in the following Java class:

```
public class Car {
    private String make;
    private int year;
}
```

Here, `make` and `year` are fields in the `Car` class. The JVM will describe each field using the `field_info` structure in the class file.

```
field_info {
  u2 access_flags; // public,private,protected,
static,final,volatile,transient,synthetic,enum
  u2 name_index;
  u2 descriptor_index; // see next slide
  u2 attributes_count;
  attribute_info attributes[attributes_count];
                          // e.g., ConstantValue
}
```

Index fields are indices into the constant pool.

Field and method descriptors

- Field descriptors describe field types
  - Base types: B - byte; C - char; D - double; F - float; I - int; J - long; S - short; Z - boolean
- Object types: L<classname>;
  - Example
    - **Descriptor-** Ljava/lang/String; **Java Type-** String **Explanation-** String name;
- Array types [ComponentType
  - Example:
    - **Descriptor-** [i **Java Type** – int[] **Explanation-** An array of integers (I).

- Method descriptors describe a method signature:
  - **(ParameterDescriptor*) ReturnDescriptor** where each is a field type, or V (void) for return.
  - Example:

    Object mymethod(int i, double d, Thread t) has
    descriptor (IDLjava/lang/Thread;)Ljava/lang/Object;

Methods in class files

```
method_info {
    u2 access_flags;
    u2 name_index;
    u2 descriptor_index;
    u2 attributes_count;
    attribute_info attributes[attributes_count];
}
```

- **access_flags**: Describes access modifiers like public, private, static, final, etc.
- **name_index**: An index into the constant pool pointing to the method's name (e.g., "myMethod").
- **descriptor_index**: An index into the constant pool pointing to the method's **descriptor**, which describes its parameter types and return type.
- **attributes_count**: The number of attributes associated with the method.
- **attributes[]**: A list of **attributes** that provide additional information about the method (e.g., bytecode, exceptions, etc.).

Bytecodes: Code of a method is held in its Code attribute

```
Code_attribute {
    // Standard Attribute Header
    u2 attribute_name_index;
    u4 attribute_length;

    // Execution Parameters (Memory Management)
    u2 max_stack;   // max number of words on the operand stack
    u2 max_locals;  // number of local variables needed

    // The Actual Bytecode Instructions
    u4 code_length;
    u1 code[code_length]; // bytecodes live here

    // Exception Handling Table
    u2 exception_table_length;
    {
        // Structure of a single exception handler entry
        u2 start_pc;   // start of try block (inclusive)
        u2 end_pc;     // end of try block (exclusive)
        u2 handler_pc; // program counter to jump to if exception is
caught
```

- The **Code_attribute** stores the **bytecode** for the method and provides information on how the method interacts with the **operand stack** and **local variables** during execution.
- It also contains an **exception table** that specifies how the method handles exceptions, and
- **Additional attributes** that provide further metadata about the method (such as exception types or code optimizations).

Example

```
public int add(int a, int b) {
    return a + b;
}
```

- **name_index**: Points to "add" in the constant pool.
- **descriptor_index**: Points to (II)I in the constant pool (method signature: two int parameters and int return type).
- **access_flags**: Contains flags for public and possibly static (if the method is static).
- **attributes_count**: The method has **1 attribute** (the Code attribute).
- **Code Attribute**:
  - **max_stack**: **2** (for the two integer operands).
  - **max_locals**: **3** (for a, b, and the result).
  - **code_length**: Depends on the bytecode, but it's relatively small (typically 3-5 bytes).
  - **code[]**: The actual bytecode (e.g., iload_1, iload_2, iadd, ireturn).
  - **exception_table_length**: **0** (no exception handling).
  - **attributes_count**: **0 or 1** (one attribute for the Code).

The bytecode ISA

- Arithmetic. Example: **`iadd`**: Adds two `int` values.
- Memory (locals, fields, statics, arrays): access and object creation, synchronization. Example: **`iload`**: Loads an `int` value from a local variable.
- Stack manipulation. Example: **`pop`**: Removes an item from the operand stack.
- Type checking and conversion. Example: **`i2f`**: Converts an `int` to a `float`.
- Control (comparisons and branches, method call and return, exceptions). Example: **`ifeq`**: Branches if the top of the stack is `0` (equivalent to `if (x == 0)`).

Intrinsics Definition in the JVM

- Intrinsics are **low-level operations** that the JVM implements in **native code** (C, C++, etc.) rather than in Java itself.
- Some operations require direct access to memory or system resources, which Java cannot do efficiently.
- They ensure fast execution of essential operations (e.g., memory management, object handling).
- **Examples of Intrinsics**:
  - **`Object.getClass()`**: Retrieves the class type of an object.
  - **`Object.clone()`**: Creates a shallow copy of an object.
  - **`Object.hashCode()`**: Returns the default hash code for an object.
  - **`System.identityHashCode()`**: Returns the identity hash code (ignores custom `hashCode()` implementations).

Native libraries

- Typically for platform-dependent code (i.e., the platform below the JVM — POSIX/Win32/..., etc.)
- Example:
  - The `RandomAccessFile` class provides native methods for performing efficient file I/O operations (read, write, seek, and file pointer management) using native code, throwing `IOException` for file access errors.
- Also typically written in the JVM implementation language

```
package java.io;
public class RandomAccessFile implements DataOutput,
    DataInput {
    ...
    public native void open(String name, boolean writable)
    throws IOException;
    public native int readBytes(byte[] b, int off, int len) throws
    IOException;
    public native void writeBytes(byte[] b, int off, int len)
    throws IOException;
    public native long getFilePointer() throws IOException;
    public native void seek(long pos) throws IOException;
    public native long length() throws IOException;
    public native void close() throws IOException;
}
```

JNI: The Java Native Interface

- This allows programmers to call out (or call back) from external libraries (written in some other, compiled language, such as C)
  - Declare native methods in your class
  - Generate a C header file using javah
  - Include <jni.h> and the generated header in your C code
  - Write wrappers in C for the functions declared in (1)
  - Compiled your C into a library
  - Load it in Java using System.loadLibrary(String)
  - Call away!
- Also includes ways to access C data, and for C to call into to the JVM (to access Java objects,create instances, etc.)

Thank You